

1.ConcurrentExclusiveSchedulerPair

1.1 .ctor

C#中我们可以通过ConcurrentExclusiveSchedulerPair来自定义调度器Scheduler，从而实现高级功能，例如控制并发数。

```
1 public class ConcurrentExclusiveSchedulerPair
2 {
3
4     private const int UNLIMITED_PROCESSING = -1;
5     /// <summary>Constant used for m_processingCount to indicate that an
    exclusive task is being processed.</summary>
6     private const int EXCLUSIVE_PROCESSING_SENTINEL = -1;
7     /// <summary>Default MaxItemsPerTask to use for processing if none is
    specified.</summary>
8     private const int DEFAULT_MAXITEMSPERTASK = UNLIMITED_PROCESSING;
9     /// <summary>Default MaxConcurrencyLevel is the processor count if not
    otherwise specified.</summary>
10    private static int DefaultMaxConcurrencyLevel => Environment.ProcessorCount;
11
12    public ConcurrentExclusiveSchedulerPair() :
13        this(TaskScheduler.Default, DefaultMaxConcurrencyLevel,
14        DEFAULT_MAXITEMSPERTASK)
15    { }
16
17    /// <summary>
18    /// Initializes the ConcurrentExclusiveSchedulerPair to target the specified
19    scheduler.
20    /// </summary>
21    /// <param name="taskScheduler">The target scheduler on which this pair should
22    execute.</param>
23    public ConcurrentExclusiveSchedulerPair(TaskScheduler taskScheduler) :
24        this(taskScheduler, DefaultMaxConcurrencyLevel, DEFAULT_MAXITEMSPERTASK)
25    { }
26
27    /// <summary>
28    /// Initializes the ConcurrentExclusiveSchedulerPair to target the specified
29    scheduler with a maximum concurrency level.
30    /// </summary>
```

```

27  /// <param name="taskScheduler">The target scheduler on which this pair should
    execute.</param>
28  /// <param name="maxConcurrencyLevel">The maximum number of tasks to run
    concurrently.</param>
29  public ConcurrentExclusiveSchedulerPair(TaskScheduler taskScheduler, int
    maxConcurrencyLevel) :
30      this(taskScheduler, maxConcurrencyLevel, DEFAULT_MAXITEMSPERTASK)
31  { }
32
33  /// <summary>
34  /// Initializes the ConcurrentExclusiveSchedulerPair to target the specified
    scheduler with a maximum
35  /// concurrency level and a maximum number of scheduled tasks that may be
    processed as a unit.
36  /// </summary>
37  /// <param name="taskScheduler">The target scheduler on which this pair should
    execute.</param>
38  /// <param name="maxConcurrencyLevel">The maximum number of tasks to run
    concurrently.</param>
39  /// <param name="maxItemsPerTask">The maximum number of tasks to process for
    each underlying scheduled task used by the pair.</param>
40  public ConcurrentExclusiveSchedulerPair(TaskScheduler taskScheduler, int
    maxConcurrencyLevel, int maxItemsPerTask)
41  {
42      ArgumentNullException.ThrowIfNull(taskScheduler);
43
44      if (maxConcurrencyLevel == 0 || maxConcurrencyLevel < -1) throw new
    ArgumentOutOfRangeException(nameof(maxConcurrencyLevel));
45      if (maxItemsPerTask == 0 || maxItemsPerTask < -1) throw new
    ArgumentOutOfRangeException(nameof(maxItemsPerTask));
46
47      // Store configuration
48      m_underlyingTaskScheduler = taskScheduler;
49      m_maxConcurrencyLevel = maxConcurrencyLevel;
50      m_maxItemsPerTask = maxItemsPerTask;
51
52      // Downgrade to the underlying scheduler's max degree of parallelism if
    it's lower than the user-supplied level
53      int mcl = taskScheduler.MaximumConcurrencyLevel;
54      if (mcl > 0 && mcl < m_maxConcurrencyLevel) m_maxConcurrencyLevel = mcl;
55
56      // Treat UNLIMITED_PROCESSING/-1 for both MCL and MIPT as the biggest
    possible value so that we don't
57      // have to special case UNLIMITED_PROCESSING later on in processing.

```

```

58     if (m_maxConcurrencyLevel == UNLIMITED_PROCESSING) m_maxConcurrencyLevel =
        int.MaxValue;
59     if (m_maxItemsPerTask == UNLIMITED_PROCESSING) m_maxItemsPerTask =
        int.MaxValue;
60
61     // Create the concurrent/exclusive schedulers for this pair
62     m_exclusiveTaskScheduler = new ConcurrentExclusiveTaskScheduler(this, 1,
        ProcessingMode.ProcessingExclusiveTask);
63     m_concurrentTaskScheduler = new ConcurrentExclusiveTaskScheduler(this,
        m_maxConcurrencyLevel, ProcessingMode.ProcessingConcurrentTasks);
64 }
65
66 }

```

- TaskScheduler: Task的调度器, 决定如何执行Task, 默认值为Default
- maxConcurrencyLevel: 最多有多少个任务可同时执行。默认值为
Environment.ProcessorCount
- maxItemsPerTask: 默认值为-1, -1时则当前线程会将任务都执行完毕之后才放回线程池, 如果设置为其他数字, 例如10, 则线程会在连续处理完10个任务之后去处理“资源饥饿”问题, 除非对于任务量极大且极其密集, 害怕霸占整个服务器的 CPU的任务, 否则大部分情况下保持-1的值可以减少线程上下文切换的开销。

```

1 public class CustomScheduler(int maxConcurrencyLevel, int maxItemsPerTask)
2 {
3
4     private readonly ConcurrentExclusiveSchedulerPair _pair =
        new(TaskScheduler.Default, maxConcurrencyLevel, maxItemsPerTask);
5
6     public Task StartConcurrent(Action<object?> action, object? state = null,
        Cancellation_token cancellationToken = default)
7     {
8
9         return Task.Factory.StartNew(action, state, cancellationToken,
            TaskCreationOptions.DenyChildAttach |
10            TaskCreationOptions.RunContinuationsAsynchronously,
            _pair.ConcurrentScheduler);
11
12     }

```

```
13
14     public Task StartConcurrent(Action action, CancellationToken
cancellationToken = default)
15     {
16
17         return Task.Factory.StartNew(action, cancellationToken,
TaskCreationOptions.DenyChildAttach |
18             TaskCreationOptions.RunContinuationsAsynchronously,
_pair.ConcurrentScheduler);
19
20     }
21
22     public Task<TResult> StartConcurrent<TResult>(Func<object?, TResult> action,
23         object? state = null, CancellationToken cancellationToken = default)
24     {
25
26         return Task.Factory.StartNew(action, state, cancellationToken,
TaskCreationOptions.DenyChildAttach |
27             TaskCreationOptions.RunContinuationsAsynchronously,
_pair.ConcurrentScheduler);
28
29     }
30
31     public Task StartExclusive(Action action, CancellationToken
cancellationToken = default)
32     {
33
34
35         return Task.Factory.StartNew(action, cancellationToken,
TaskCreationOptions.DenyChildAttach |
TaskCreationOptions.RunContinuationsAsynchronously,
36             _pair.ExclusiveScheduler);
37
38     }
39
40     public Task StartExclusive(Action<object?> action, object? state = null,
CancellationToken cancellationToken = default)
41     {
42
43
44         return Task.Factory.StartNew(action, state, cancellationToken,
TaskCreationOptions.DenyChildAttach |
TaskCreationOptions.RunContinuationsAsynchronously,
```

```

45         _pair.ExclusiveScheduler);
46
47     }
48
49     public Task<TResult> StartExclusive<TResult>(Func<object?, TResult> func,
50     object? state = null, CancellationToken cancellationToken = default)
51     {
52
53         return Task.Factory.StartNew(func, state, cancellationToken,
54         TaskCreationOptions.DenyChildAttach |
55         TaskCreationOptions.RunContinuationsAsynchronously,
56         _pair.ExclusiveScheduler);
57
58     }
59 }

```

1.2 ConcurrentScheduler&ExclusiveScheduler

ConcurrentScheduler&ExclusiveScheduler的本质是一个Reader-Writer Lock Model（读写锁模型），我们可以从3个方面对其进行深入理解：

- ConcurrentScheduler本质就是多线程并发运行，上限就是等于传入的 maxConcurrencyLevel；
- ExclusiveScheduler本质就是单线程串行运行；
- ConcurrentScheduler和ExclusiveScheduler本质是互斥的，它们不是独立的，当 ExclusiveScheduler的单线程任务在运行时，ConcurrentScheduler的所有并发任务都会被挂起；反过来，只有当ConcurrentScheduler多线程任务运行完毕之后，ExclusiveScheduler里的队列任务才会开始执行，所以本质是一个读写锁模型在调度器的映射和实现。

1.3 Scheduler Chaining

既然我们可以自定义Scheduler，那么当然可以将自定义的Scheduler作为第一个参数传入后续的自定义Scheduler中，这样就产生了C#高级编程的一个技巧，即调度器链，当我们

把一个自定义调度器 A作为参数传给另一个自定义调度器B的构造函数时，实际就构建了一个**分层调度网络**。分层调度器的作用和信号量类似，主要用来进行限流。例如，现在有这样的需求：由于CPU的限制，最多只能同时跑10个线程，否则会影响其他的核心业务，其中大模型的AI推理程序最多只能占用2个线程，防止显存溢出，可以分别用信号量 SemaphoreSlim和调度链来实现。

```
1 public class SemaphoreGateway
2 {
3     private readonly SemaphoreSlim _globalLimit = new(10, 10);
4     private readonly SemaphoreSlim _aiLimit = new(2, 2); //new (1,1)
5
6     public async Task RunAiInferenceAsync(Func<Task> aiWork)
7     {
8
9         await _aiLimit.WaitAsync();
10        try
11        {
12            await _globalLimit.WaitAsync();
13            try
14            {
15
16                await Task.Run(worker);
17            }
18            finally
19            {
20                _globalLimit.Release();
21            }
22        }
23        finally
24        {
25            _aiLimit.Release();
26        }
27    }
28 }
```

```

1 public class SchedulerGateway
2 {
3     private readonly ConcurrentExclusiveSchedulerPair _rootGateway =
4         new(TaskScheduler.Default, maxConcurrencyLevel: 10);
5
6     private readonly ConcurrentExclusiveSchedulerPair _aiSchedulerPair;
7
8     public SchedulerGateway()
9     {
10         _aiSchedulerPair = new
11             ConcurrentExclusiveSchedulerPair(_rootGateway.ConcurrentScheduler,
12             maxConcurrencyLevel: 2);
13     }
14
15     public Task RunAiInferenceAsync(Action work) =>
16         Task.Factory.StartNew(work, CancellationTokens.None,
17             TaskCreationOptions.None, _aiSchedulerPair.ConcurrentScheduler);
18 }

```

现在当我们向_aiSchedulerPair提交了50个任务时，由于超过了最大并发数2，它会将2个任务向上层提交，剩下的48个留在队列里排队；当_rootGateway收到之后，如果当前还有空闲的线程（设置的为10），则会继续向C#线程池提交，并执行任务；如果_rootGateway的总任务达到了10个之后，就会停止接收底层的任务提交，起到限流的作用。在SchedulerGateway里面我们还有添加各种自定义的调度器，例如可以添加一个写数据库的调度器，代码如下。

```

1 private readonly ConcurrentExclusiveSchedulerPair _dbSchedulerPair;
2
3 ///....
4 _dbSchedulerPair = new
5     ConcurrentExclusiveSchedulerPair(_rootGateway.ConcurrentScheduler,
6     maxConcurrencyLevel: 1);

```

因此，我们可以看出，Scheduler Chaining的本质是**控制反转和资源池隔离**，和信号量相比，它的优势可以总结为以下几点：

- 避免代码入侵：不需要写大量的try-catch-finally，所有的拦截和转发全部在底层的Scheduler里面完成，外层只需要调用RunAllInferenceAsync方法即可；
- 如果想用Semaphore实现独占任务，则会很麻烦，而使用调度器时直接使用ExclusiveScheduler即可（给大家提醒一下Semaphore(1,1)不是读写锁，这个时候如果想用读写锁，最佳方案是用Stephen Cleary大神写的Nito.AsyncEx中的AsyncReaderWriterLock)
- Scheduler可以进行环境继承，避免越狱问题。

最佳实践：I/O密集型，如Mysql连接数限制、HTTP并发请求限制等用SemaphoreSlim；CPU密集型，并且希望将限流逻辑”和“业务计算逻辑”彻底剥离开来，此时用调度链。

2.Scheduler.Current（内聚）

首先，需要明确的是Current的使用场景为：**环境继承及上下文传递**。之前说过，Current会使用当前的调度器来执行Task，而这个调度器本质取决于当前代码运行的环境，例如如果是在普通的Task里面，则是C#的线程池；如果是WPF或者Winform，则是UI相关的上下文调度器；如果是自定义的调度器，则Current就是调度器。对于自定义调度器的情形，假设我们已经设置了最大并发度为2，如果我们在调度器的内部还创建了Task，如果直接用Task.Run的话，如下所示：

```
1 var aiTask = Task.Factory.StartNew(() =>
2 {
3     Task.Run(() => ComputeMatrixA());
4     Task.Run(() => ComputeMatrixB());
5     Task.Run(() => ComputeMatrixC());
6 }, ..., myLimitedScheduler);
```

此时Task.Run会直接使用默认的线程池来执行，那么自定义的调度器完全就被击穿了，没有起到任何作用，但如果我们使用StartNew来创建的话，由于默认的规则为Current，则由于外层处于限流调度器中，所以这内部子任务也会乖乖进入限流队列排队，如下所示：

```
1 var aiTask = Task.Factory.StartNew(() =>
```



```

2 {
3     Task.Factory.StartNew(() => ComputeMatrixA());
4     Task.Factory.StartNew(() => ComputeMatrixB());
5     Task.Factory.StartNew(() => ComputeMatrixC());
6 }, ..., myLimitedScheduler);

```

3.HideScheduler（解耦）

HideScheduler的核心思想是**防御性编程**，作用是阻断自定义调度器的向下传播，保护系统的纯洁性，当我们调用第三库的时候，如果相关API内部使用了100个StartNew方法且默认指定Scheduler为Current，那么这种情况下就会直接将自定义的限流调度器给占满，导致核心业务直接卡死，为了应对这种情况，就需要使用HideScheduler，它会强行切断上下文，使用默认的线程池来执行任务

```

1 Task.Factory.StartNew(
2     () => {
3         // 它看到的 TaskScheduler.Current 不再是外层的调度器，
4         // 而是会被强制重置为 TaskScheduler.Default（全局线程池）
5         //ThirdParty.ApiAsync();
6         Task.StartNew();
7         Task.StartNew();
8         Task.StartNew();
9         //...
10    },
11    CancellationToken.None,
12    TaskCreationOptions.HideScheduler,
13    myLimitedScheduler);

```

总结：HideScheduler核心就是既要用自定义调度器限制最外层的并发总数，又要防止自定义调度器被内部的黑盒代码向下传染

4.UnobservedTaskException

如果第三方库其内部只是写了类似于Task.Run(() =>())这样的代码，没有将Task返回给调用方，而是直接return并结束了方法，并且抛出了异常，则可能会出现以下情况：

- 外层代码await瞬间执行完毕；
- 外层的try/catch/finally代码块执行完毕；
- 过了一段时间，第三方代码执行出现异常，一般称之为UnobservedTaskException，这个时候会出现两种情况：

- (1) .Net4.5之前，GC会直接杀死宿主进程；
- (2) .Net4.5之后，会吞掉这个异常，不做任何处理；

为了避免这种情况，一般都会对TaskScheduler添加一个全局的监视方法

UnobservedTaskException,代码如下：

```
1 TaskScheduler.UnobservedTaskException += (sender, args) =>
2 {
3
4     Log.Error("捕获到未处理的异常:", args.Exception);
5
6     args.SetObserved();
7
8
9 };
```

当然，也可以在内部直接用try代码块进行捕获，如下所示：

```
1 public void DoWorkSafely()
2 {
3     Task.Run(() =>
4     {
5         try
6         {
7             ThirdParty.ApiAsync();
8         }
9         catch (Exception ex)
10        {
11            //...
12        }
13    });
14 }
```

```

1 public async Task DoWorkAndWaitAsync()
2 {
3
4     try
5     {
6         await Task.Run(() =>
7         {
8             ThirdParty.ApiAsync();
9         });
10    }
11    catch (Exception ex)
12    {
13        //...
14    }
15 }

```

一定要注意禁止写出以下的异常捕获代码：

```

1 public async Task DoWorkAndWaitAsync()
2 {
3
4     try
5     {
6         Task.Run(() =>
7         {
8             ThirdParty.ApiAsync();
9         });
10    }
11    catch (Exception ex)
12    {
13        //...
14    }
15 }

```

最佳实践：只要起了一个异步操作，你要么await它，要么把它的引用保存进List<Task>然后Task.WhenAll，永远绝对不要写出丢后不管的代码！